

Rapport de Projet



2022 – 2023



SmartInvest

Création d'une plateforme
Immobilière/d'investissement :

SmartInvest

Table des matières

I.	Contexte	3
II.	Cahier des charges	4
1.	Présentation de l'équipe	4
2.	Contrainte de développement	5
a.	Objectifs	5
3.	Planification.....	6
a.	Délai de réalisation	6
b.	Répartition des tâches.....	6
c.	Diagramme de classes	7
4.	Outils et langages	9
	JSON Web Token	10
	Relation entre base de données et client.....	11
III.	Développement du site	14
1.	Les utilisateurs.....	15
2.	Les Listes.....	18
A.	Création des listes	18
B.	Affichage des listes	19
C.	Interface d'une liste.....	20
E.	Les opérations des listes.....	22
3.	Calcul du taux d'endettement	24
4.	Simulateur de crédit	25
IV.	Conclusion	26
	Bibliographie	27

I. Contexte

ImmoMetz est une agence immobilière située au centre-ville de Metz qui a pour principale activité l'achat et la vente de biens immobiliers et la mise en location de logements pour des colocations. Sa politique est centrée sur le fait d'aider les primo-accédants à trouver le bien qui leur correspond et plus généralement d'aider la jeunesse à accéder à un logement.

Constatant une baisse de l'investissement immobilier parmi ses jeunes clients due, entre autres, aux lourdeurs administratives et au manque d'informations, l'entreprise ImmoMetz nous a mandatés pour créer une application ayant pour but d'aider ses futurs clients à investir.

L'objectif est donc de créer une application possédant divers outils qui serviront aux futurs acquéreurs pour calculer leurs emprunts avec le taux et la durée les plus justes et les plus adaptées à leur projet.

Cette application a également pour but d'être un outil de confiance dans lequel le futur acquéreur pourra accéder à différents articles sur l'immobilier et l'actualité concernant l'application et l'agence.

ImmoMetz nous a également demandé que cette application embarque un système de listes de dépenses partageables pour gérer des budgets à plusieurs destinés aux colocations gérées par l'agence.

Chaque membre de la liste pourra donc ajouter de l'argent pour les différents projets/budgets de la colocation ou, pour un groupe d'amis, prévoir différentes sorties.

Il est nécessaire que cette application soit facile à utiliser et donne envie à l'utilisateur de venir chercher l'information.

II. Cahier des charges

1. Présentation de l'équipe

	Rôles	Noms
Développeur	<i>Déploiement de l'application et relation entre l'api et vue.js :</i> - Visual Studio Code - Vue.js - Symfony	 Grégoire Larrans Fondateur / Développeur SmartInvest
Développeur	<i>Développeur vue.js et SQL Manager :</i> - Visual Studio Code - SQL - Vue.js	 Tristan Lhotellier Fondateur / Développeur SmartInvest
Développeur	<i>Mise en place de l'api de l'application et administrateur système :</i> - PhpStorm - Symfony - Linux	 Julien Delvaux Fondateur / Développeur SmartInvest
Graphiste	<i>Création des templates du site et développeur frontend :</i> - Penpot - Quasar	 Kemil Djermoune Fondateur / Développeur SmartInvest

2. Contrainte de développement

La mise en place des fonds partagés est une contrainte par la complexité de créer des listes partageables et éditables par tous. Mais également, le fait de gérer la mise à jour des listes avec plusieurs utilisateurs est un défi car il faut que la liste reste disponible et qu'elle soit à jour.

L'adaptabilité de l'application sur différentes plateformes est une contrainte à prendre en compte étant donné la large gamme d'appareils présents sur le marché, il n'est pas possible de négliger une plateforme qui empêcherait un certain nombre d'utilisateurs de profiter de l'application.

Il est également important que l'interface soit simple à comprendre et intègre toutes les fonctionnalités sans les surcharger, il faut que l'application soit compréhensible au premier coup d'œil et facile d'utilisation.

Au vu de l'actualité en matière de cybersécurité, le traitement des données des utilisateurs est une priorité au vu de l'importance de ces dernières, il est primordial pour nous de mettre l'accent sur la confidentialité.

De plus, ImmoMetz nous impose d'utiliser les mêmes technologies présentes au sein de leur entreprise afin de permettre la compatibilité entre les différents services proposés, c'est-à-dire VueJs, Quasar, Apache et NodeJs.

Afin d'estimer le coût au plus juste de cette application, nous nous sommes basés sur une durée de 5 mois en prenant en compte le développement de l'application et de ses fonctionnalités, les tests unitaires et enfin le déploiement de l'application. Les coûts sont donc estimés à environ 15 000 euros.

a. Objectifs

- Connaître les actualités en investissement immobilier
- Avoir une application intuitive et simple d'utilisation
- Rechercher rapidement des textes de lois relatifs à l'investissement
- Calculer un taux d'endettement
- Calculer un emprunt avec un taux et une durée
- Estimer le coût d'un emprunt
- Créer et partager des listes de budget
- Éditer et modifier des listes de budget

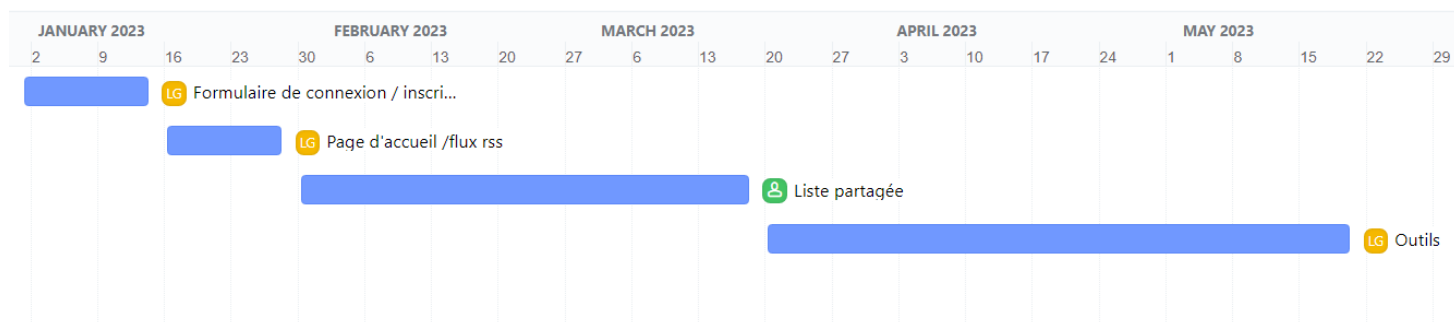
3. Planification

a. Délai de réalisation

Les délais sont fixés par le centre de formation **Mewo**. Le cahier des charges était prévu initialement pour le 14 décembre 2022, le rendu du rapport de projet devra se faire au plus tard le **19 juin 2023**.

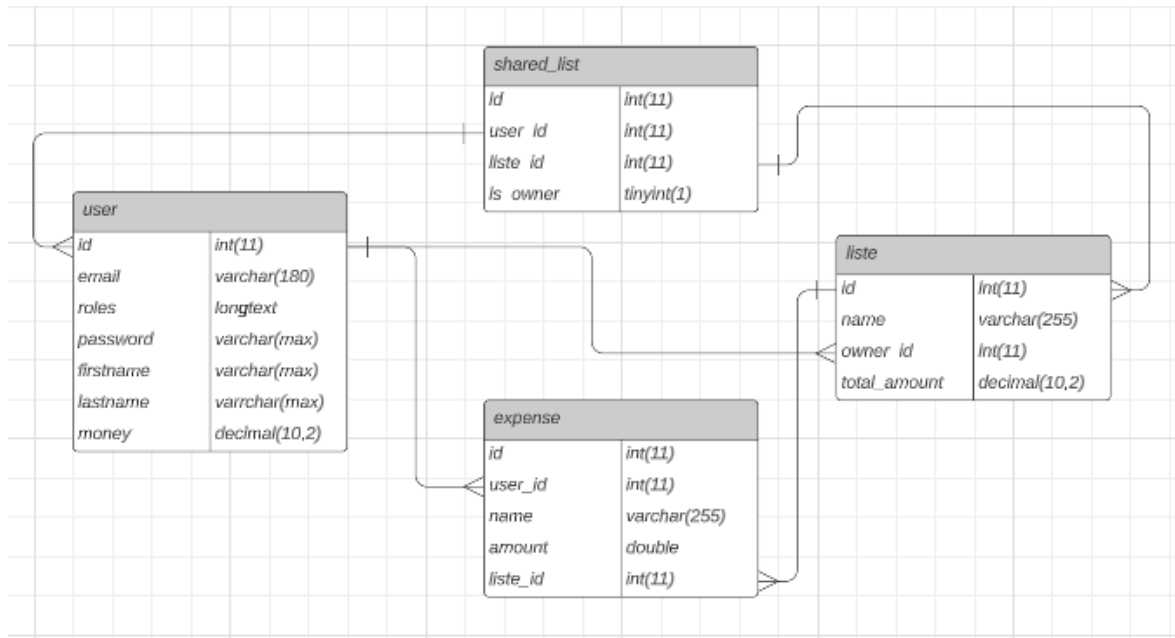
b. Répartition des tâches

Not started	ACTION REQUIRED	SCHEDULE	PRIORITY	STATUS	ASSIGNED TO	ALL USERS	+
Formulaire de connexion / inscription	LG Larrans Gregoire		Normal	Not started	LG Larrans Gregoire	LG JD	
Page d'accueil /flux rss	LG Larrans Gregoire		Normal	Not started	LG Larrans Gregoire	LG JD	
Liste partagée 3	TL Tristan Lhotellier		Normal	Not started	JD Kemil Djermoune	TL JD	
Ajouter des gens	JD Kemil Djermoune		Normal	Not started	JD Kemil Djermoune	JD	
Créer un fond commun	JD Kemil Djermoune		Normal	Not started	TL Tristan Lhotellier	TL JD	
Partager le budget	TL Tristan Lhotellier		Normal	Not started	TL Tristan Lhotellier	TL JD	
Outils 3	LG Larrans Gregoire		Normal	Not started	LG Larrans Gregoire	LG JD	
Page calculateur taux d'endettement	LG Larrans Gregoire		Normal	Not started	JD Julien Delvaux	LG JD	
Taux d'emprunt	LG Larrans Gregoire		Normal	Not started	LG Larrans Gregoire	LG JD	
Simulateur de crédit	LG Larrans Gregoire		Normal	Not started	JD Julien Delvaux	LG JD	



c. Diagramme de classes

Dans notre projet, le diagramme de classes peut être utilisé pour représenter les différentes entités du système, que nous verrons dans un instant. Il permet de clarifier la structure du système et de mieux comprendre les relations entre les différentes entités, ce qui facilite la conception d'un système plus robuste et évolutif. De plus, il peut servir de documentation pour le système et faciliter la communication et la collaboration entre les membres de l'équipe de développement. Ci-dessous le diagramme de classe de notre base de données :



On peut voir que nous avons 4 tables : user, shared_list, liste et expense avec différentes relations. La table user a une relation Many-to-many. Pour ce faire, nous avons créé une table tampon appelée Shared_list qui a deux relations one-to-many vers la table User et la table Liste.

Pour sauvegarder les opérations de l'utilisateur, il était nécessaire d'ajouter une table expense qui a une relation avec les tables user et liste.

Description des classes :**Classe User :**

La classe User représente un utilisateur. Elle possède les attributs suivants :

- Id : un identifiant unique pour chaque utilisateur
- Email : le mail de l'utilisateur
- roles : concerne les autorisations des utilisateurs
- Password : son mot de passe
- First_name : son prénom
- Last_name : son nom de famille
- Money : le montant de son argent

Classe Shared_list :

La classe Shared_list représente les utilisateurs sur une liste donnée grâce à l'id de cette dernière. Elle possède les attributs suivants :

- Id : un identifiant unique pour chaque utilisateur
- User_id : l'id d'un utilisateur
- Liste_id : l'id d'une liste
- is_owner: un booléen pour savoir si l'id lié à l'utilisateur est le même que celui qui a créé la liste

Classe Liste:

La classe Liste représente une liste. Elle possède les attributs suivants :

- Id : un identifiant unique pour chaque emprunt
- Name : le nom de la liste
- Owner_id : l'id de l'utilisateur qui est le créateur de la liste
- Total_amount : le montant total de la liste

Classe Expense :

La classe Expense représente une opération par un utilisateur. Elle possède les attributs suivants :

- Id : un identifiant unique pour chaque dépense
- User_id : l'id de l'utilisateur qui a effectué l'opération
- Amount : le montant de l'opération
- Liste_id : l'id de la liste liée à l'opération

4. Outils et langages

En ce qui concerne les environnements de développement, trois d'entre nous utilisent principalement Visual Studio Code et un seul utilise PHP Storm.



Pour le projet, nous utilisons 2 principaux Framework. Il s'agit de Quasar avec VueJs3 pour la partie utilisateur, et pour l'api nous utilisons Symfony 5.4



Les langages utilisés sont PHP 7.4, Type Script, JavaScript et HTML5.



Pour les tests côté api nous nous sommes servis de Postman et pour installer les package, l'incontournable composer.



Afin de mutualiser le travail, nous avons utilisé deux solutions :

- Nous avons utilisé GitHub et GitHub Desktop pour travailler sur une branche de développement et ensuite, après vérifications, nous fusionnons les changements dans la branche principale.
- Nous avons également loué un serveur chez OVH, ce qui nous permet d'avoir notre travail en production. Cela nous permet de vérifier que le travail que chacun fait est viable dans l'environnement de production.
- Pour mettre à jour l'API, qui n'est pas stockée sur GitHub, nous avons vérifié que les différentes fonctionnalités ajoutées fonctionnaient correctement, puis nous avons utilisé Filezilla pour mettre à jour les fichiers.



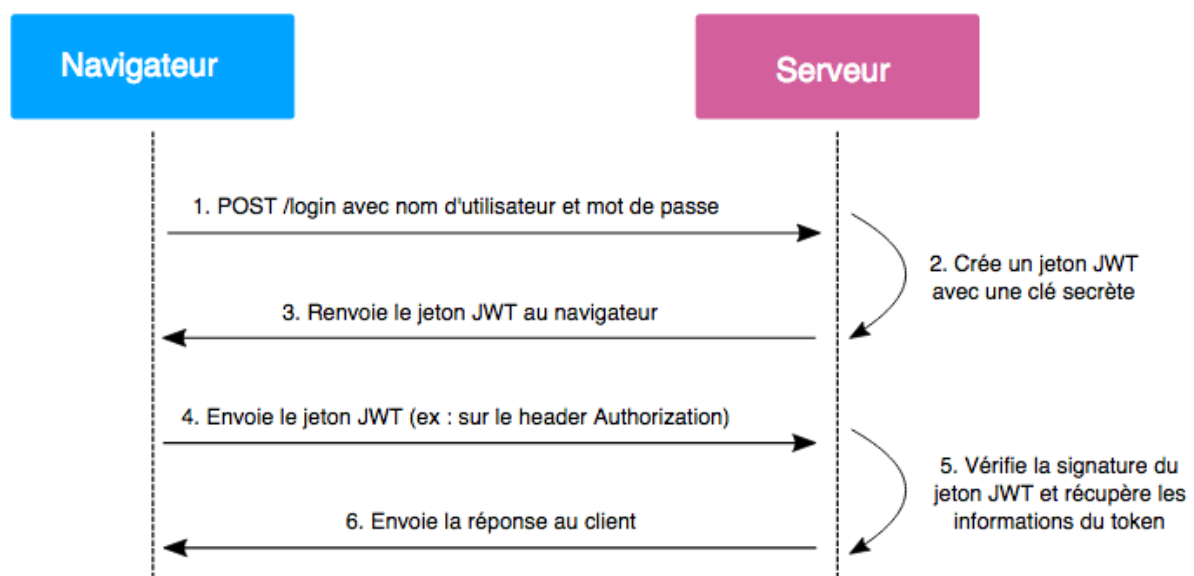
JSON Web Token

Comme nous avons également une application mobile à prendre en compte, la création d'une API nous a semblé un choix pertinent pour nous faciliter la tâche. Nous avons donc pris la décision d'utiliser le framework Symfony car il permet de créer une API facilement et rajoute une couche sécurité grâce au bundle Doctrine.

Nous avons aussi intégré l'authentification JWT pour envoyer les données au client. Les « JSON Web Token » ou JWT sont des jetons générés par un serveur lors de l'authentification d'un utilisateur sur une application Web, et qui sont ensuite transmis au client.

Ils seront renvoyés avec chaque requête HTTP au serveur, ce qui lui permettra d'identifier l'utilisateur. Pour ce faire, les informations contenues dans le jeton sont signées à l'aide d'une clé privée détenue par le serveur. Quand il recevra à nouveau le jeton, le serveur n'aura qu'à comparer la signature envoyée par le client et celle qu'il aura générée avec sa propre clé privée et à comparer les résultats. Si les signatures sont identiques, le jeton est valide.

Il faut noter que le JWT est unique et ne peut être utilisé que par un seul utilisateur à la fois. Autrement dit, si un utilisateur essaye de partager son compte, un des deux utilisateurs ne pourra pas faire de requête car son jeton ne sera pas valide.



Relation entre base de données et client

Afin de pouvoir utiliser une base de données depuis une application de type web faite en VueJs3 et Quasar, nous avons fait de nombreuses recherches. Après avoir soulevé différents problèmes en matière de sécurité, notre choix s'est porté sur le framework Symfony pour réaliser une API REST.

Ce choix se justifie par ces arguments :

- Symfony est un framework qui permet de réaliser rapidement et de manière sécurisée, une API REST en PHP
- C'est un framework maîtrisé par les membres du groupe
- Il amène une surcouche de sécurité avec Doctrine pour réaliser la relation avec la base de données.
- L'utilisation d'une API REST permet une adaptabilité sur tout type de support

Pour assurer la relation entre la partie API et la partie client nous utilisons la bibliothèque javascript Axios.

Axios est souvent utilisé pour communiquer avec des API, qui sont des interfaces de programmation permettant à différentes applications de communiquer entre elles et d'échanger des données. Les API sont souvent utilisées pour récupérer des données à partir d'une source externe, comme une base de données ou un service tiers, et pour les afficher dans une application Web.

Axios est très utile pour communiquer avec des API car elle prend en charge la plupart des méthodes HTTP, telles que GET, POST, PUT et DELETE, qui sont couramment utilisées pour interagir avec les API. De plus, elle peut facilement convertir les données en JSON, qui est un format de données couramment utilisé pour les API.

De nombreux services tiers, tels que les réseaux sociaux ou les services de paiement en ligne, proposent des API pour que les développeurs puissent intégrer leurs fonctionnalités dans des applications Web. Axios facilite grandement la communication avec ces API, en offrant une API simple et facile à utiliser pour envoyer des requêtes et récupérer des données.

En somme, Axios est une bibliothèque très utile pour communiquer avec des API et récupérer des données à partir de sources externes. Elle est compatible avec de nombreux services tiers proposant des API et facilite grandement l'interaction avec ces dernières.

Ci-dessous un exemple de requête Axios avec l'utilisation d'un token. Il faut dans un premier temps utiliser la fonction get de Axios.

Nous lui passons en paramètre l'url de l'api en précisant la route de la fonction à laquelle nous souhaitons accéder.

```
const response = await axios.get('http://135.125.12.76/api/listes', {
  headers: {
    Authorization: `Bearer ${utilisateur.token}`,
  },
});
// this.liste = await response.data;
return await response.data;
} catch (error) {
  console.error(error);
}
```

Dans la requête nous utilisons le headers d'Axios qui va nous permettre de passer dans le champ Authorization le token. Afin d'intégrer le token, il faut utiliser le Bearer, qui est défini dans la partie sécurité de l'api puis mettre le token de l'utilisateur.

Ce token est stocké à partir du moment où l'utilisateur va se connecter. Une fois la requête envoyée vers l'api, nous attendons la réponse. S'il y a une réponse, nous récupérons les données (data).

Axios nous permet également de faire un catch dans la requête en cas d'erreur et d'afficher cette erreur.

```

recupAllList() {
  ...
  return axios
    .get('http://135.125.12.76/api/listes', {
      headers: {
        Authorization: `Bearer ${utilisateur.token}`,
      },
    })
    .then((response) => {
      if (response) {
        console.log('MES LISTE', response.data.listes);
        this.allList = <Liste[]>response.data.listes;
      }
    });
},

```

Dans cette fonction, nous utilisons encore le get d'Axios. De la même manière que la première fois, nous passons les différents paramètres avec le token. La différence ici est l'utilisation du then. Le « .then() » correspond à l'attente de l'exécution de la requête.

Concrètement ici nous attendons la réponse et s'il y a une réponse, nous assignons à la constante recupAllList. Les valeurs qui sont remontées dans le corps de la réponse qui sont ici nommé listes qui seront de type Liste.

```

getListById(id:number) {
  return axios.get(`http://135.125.12.76/api/liste/${id}`, {
    headers: {
      Authorization: `Bearer ${utilisateur.token}`,
    },
  });
},

```

Dans cette fonction par exemple, nous passons comme argument un id de type number. Nous devons ensuite passer l'arguments en passant l'id dans l'url de la requête. Cette id sera ensuite utilisé par l'api et utilisera sa valeur dans la requête liée à la route.

III. Développement du site

Tout d'abord nous avons créé toutes nos entités.

```
namespace App\Entity;

use ...

/** @ORM\Entity(repositoryClass="App\Repository\UserRepository") ...*/
class User implements UserInterface, PasswordAuthenticatedUserInterface
{
    public const ROLES = [...];

    /** @ORM\Id ...*/
    private ?int $id = null;

    /** @ORM\Column(type="string", length=180, unique=true) ...*/
    private ?string $email = null;

    /** @ORM\Column(type="json") ...*/
    private array $roles = [];

    /** @ORM\Column(type="string") ...*/
    private ?string $password = null;

    /** @ORM\Column(type="string") ...*/
    private ?string $firstname = null;

    /** @ORM\Column(type="string") ...*/
    private ?string $lastname = null;

    /** @ORM\Column(type="decimal", precision=10, scale=2, nullable=true) ...*/
    private ?string $money = null;

    public function getId(): ?int{...}

    public function getEmail(): ?string{...}

    public function setEmail(string $email): self{...}

    /** A visual identifier that represents this user. ...*/
    public function getUserIdentifier(): string
```

Puis nous avons fait deux controller, un pour tout ce qui concerne la partie utilisateur et un autre qui va concerner toute la partie liste.

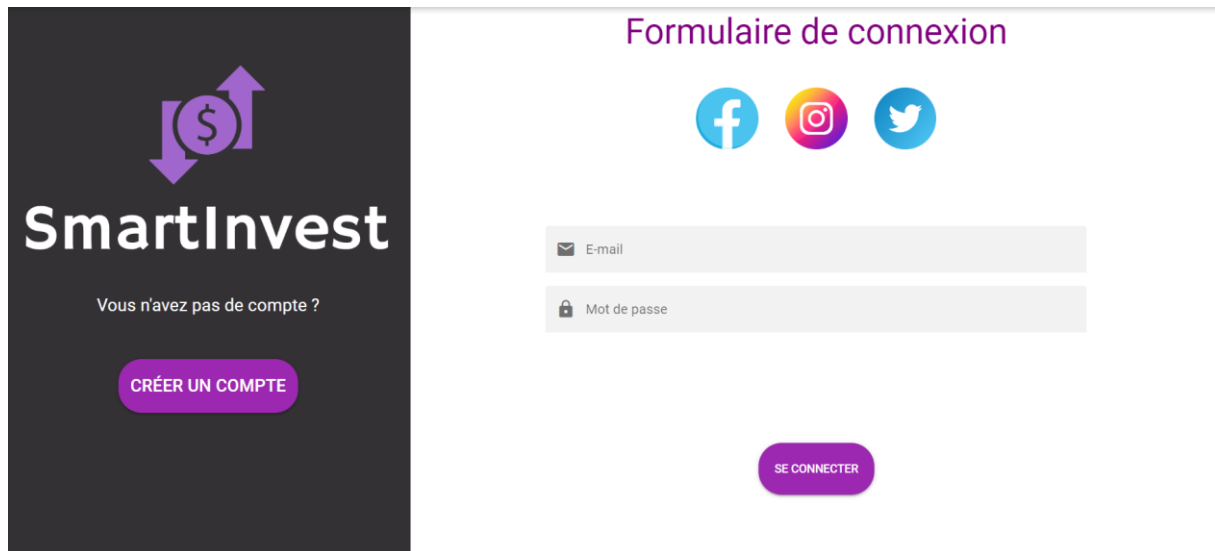
Et enfin l'ajout de JWT se fait avec la ligne de commande suivante : « *composer require firebase/php-jwt* ».

1. Les utilisateurs

On a commencé par mettre en place la création des comptes ainsi que l'authentification.

Formulaire de connexion

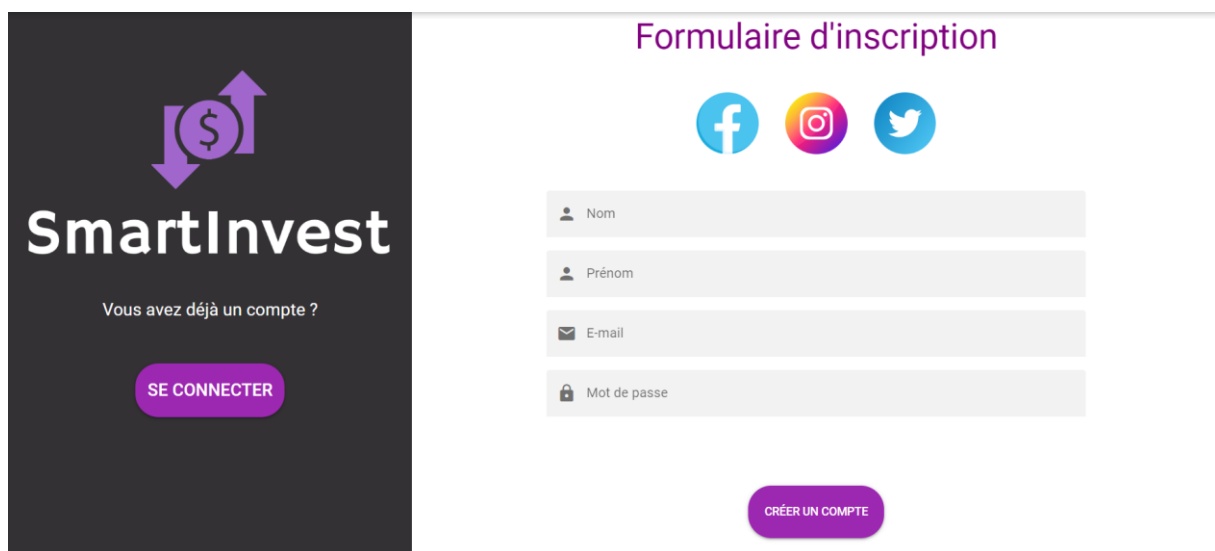
La première étape était la mise en place d'un formulaire de connexion, contenant les champs habituels email/mot de passe et un bouton permettant de se connecter, mais également un bouton permettant d'accéder à la page « Formulaire d'inscription » dans le cas où l'utilisateur ne possède pas encore de profil.



The image shows the 'Formulaire de connexion' (Login Form) for SmartInvest. On the left is a dark sidebar with the SmartInvest logo (a purple circle with a white dollar sign and two arrows) and the text 'SmartInvest'. Below the logo, it says 'Vous n'avez pas de compte ?' and there is a purple button labeled 'CRÉER UN COMPTE'. The main area is white and titled 'Formulaire de connexion' in purple. It features three social media icons (Facebook, Instagram, Twitter) at the top. Below them are two input fields: 'E-mail' and 'Mot de passe' (password), each with a corresponding icon (envelope and lock). At the bottom right of the form is a purple button labeled 'SE CONNECTER'.

Création du compte

Puis nous avons fait le formulaire d'inscription, on y retrouve les champs des informations demandées pour l'inscription et la possibilité d'accéder au formulaire de connexion si l'utilisateur possède déjà un compte.



The image shows the 'Formulaire d'inscription' (Registration Form) for SmartInvest. On the left is a dark sidebar with the SmartInvest logo and the text 'SmartInvest'. Below the logo, it says 'Vous avez déjà un compte ?' and there is a purple button labeled 'SE CONNECTER'. The main area is white and titled 'Formulaire d'inscription' in purple. It features three social media icons (Facebook, Instagram, Twitter) at the top. Below them are four input fields: 'Nom' (Name), 'Prénom' (First Name), 'E-mail', and 'Mot de passe' (password), each with a corresponding icon (person, person, envelope, and lock). At the bottom right of the form is a purple button labeled 'CRÉER UN COMPTE'.

Les icônes amenant sur les différents réseaux sociaux ne sont pas encore terminées, dans l'attente de la création des différents profils sur ces différents réseaux.

Regardons maintenant ce qui se passe en coulisse :

```
/**
 * @Route("/api/create_user", name="create_user")
 */
public function createUser(Request $request, UserRepository $userRepository, UserPasswordHasherInterface $passwordHasher): Response
{
    $content = $request->toArray();
    $user = new User();
    $user->setEmail($content['email']);
    $user->setMoney($content['money']);
    $user->setFirstname($content['firstName']);
    $user->setLastname($content['lastName']);

    $plaintextPassword = $content['password'];

    $hashedPassword = $passwordHasher->hashPassword(
        $user,
        $plaintextPassword
    );

    $user->setPassword($hashedPassword);
    $userRepository->save($user, flush: true);

    return $this->json(
        [
            'user' => 'OK'
        ]
    );
}
```

On dit à Symfony que les données sont sous forme de tableau, on crée un nouvel objet « User » puis on lui passe tous les champs attendus. Cependant, il y a une petite particularité pour le mot de passe car une fois reçu en chaîne de caractères, nous allons le hacher avant de l'enregistrer. Malheureusement, lors des tests sur le site, nous avons rencontré notre premier problème.

Nous avons rencontré des difficultés lors de l'intégration des JWT car la communication en local entre l'API et notre site s'effectuait bien, mais une fois en production, la partie frontend nous a bloqués à cause d'un problème de CORS policy.

```
✖ Access to XMLHttpRequest at 'http://135.125.12.76/api/login_check' from origin 'http://135.125.12.76:8080' has been blocked by CORS policy: Response to preflight request doesn't pass access control check: It does not have HTTP ok status.
```


On a passé un moment avant de trouver la solution mais il manquait simplement quelques lignes de configuration dans un fichier qui se nomme `nelmio_cors.yaml`.

```
nelmio_cors:
  defaults:
    allow_credentials: false
    allow_origin: []
    allow_headers: []
    allow_methods: []
    expose_headers: []
    max_age: 0
    hosts: []
    origin_regex: false
    forced_allow_origin_value: ~

  paths:
    '^/api/':
      allow_origin: ['*']
      allow_headers: ['X-Custom-Auth', 'Authorization', 'Content-type']
      allow_methods: ['POST', 'PUT', 'GET', 'DELETE', 'OPTIONS']
      max_age: 3600
    '^/':
      origin_regex: true
      allow_origin: ['^http://localhost:[0-9]+']
      allow_headers: ['X-Custom-Auth']
      allow_methods: ['POST', 'PUT', 'GET', 'DELETE', 'OPTIONS']
      max_age: 3600
      hosts: ['^api\\.']
```

Configuration ajouté

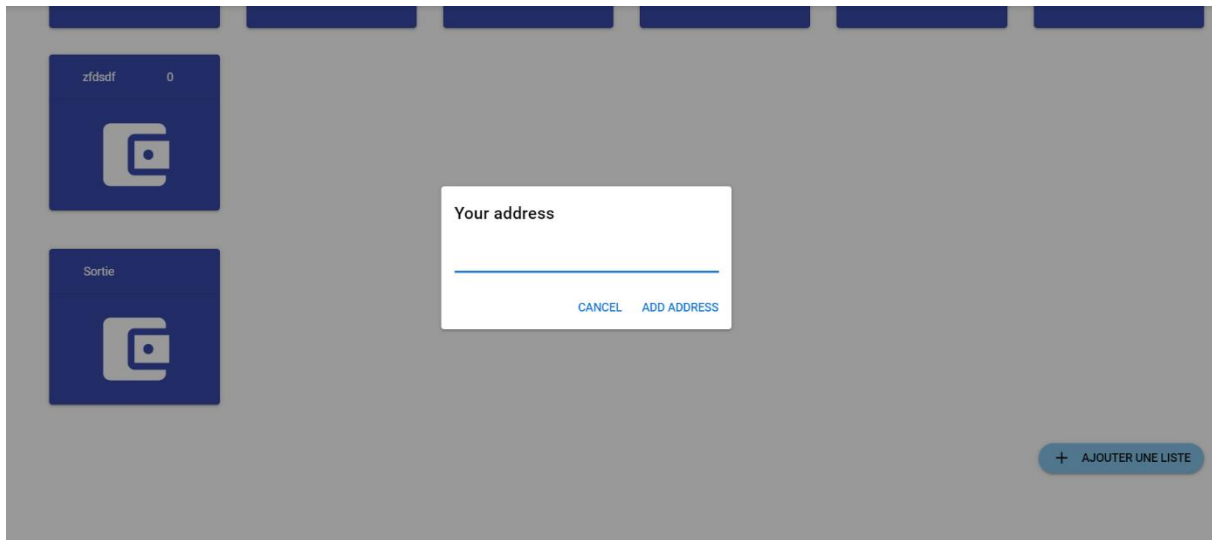
Une fois ces lignes ajoutées, notre problème était résolu et la création des comptes ainsi que la connexion étaient fonctionnelles.

2. Les Listes

A. Création des listes

Nous avons d'abord commencé par mettre en place la création des listes

La création d'une liste se fait via un pop-up, ou il suffit d'y enregistrer le nom de la liste



Donc, nous avons juste besoin de récupérer son nom, le reste des informations étant généré automatiquement (l'ID est auto-incrémenté et la propriété "owner" est déterminée par l'utilisateur qui crée la liste).

```
/**
 * @Route ("/api/liste/create", name="create_liste")
 */
public function createList(Request $request, ListeRepository $listeRepository): Response
{
    $liste = $this->serializer->deserialize($request->getContent(), type: Liste::class, format: 'json');
    $liste->setOwner($this->getUser());

    $listeRepository->add($liste, flush: true);

    return $this->json(
        [
            'liste' => 'OK'
        ]
    );
}
```

Pas de difficulté particulière ici avec le module serializer, mais nous avons rencontré un petit obstacle un peu plus tard qui concerne les références circulaires, même si mon collègue m'avait prévenu que ça nous arriverait.

B. Affichage des listes

Affichage des listes

Les listes s'affichent les unes à la suite des autres sous forme de différents porte-monnaie, ce qui permet d'avoir une vue globale de ses listes et de ne pas être perdues lors de la recherche d'une d'entre elles. Elles sont classées selon leur numéro id



Ensuite, nous avons mis en place la fonction pour afficher nos listes.

```
/**
 * @Route("/api/listes", name="app_liste")
 */
public function displayLists(): Response
{
    /** @var User $user */
    $user = $this->getUser();

    // envoi du json au front
    $data = $this->serializer->serialize($user, format: 'json', ['groups' => 'shared_list']);

    return new JsonResponse($data, status: 200, [], json: 'json');
}
```

C'est ici que le problème des références circulaires est survenu car nous faisons appel à nos entités qui ont des relations entre elles et donc boucle en permanence sur les informations à récupérer.

Pour corriger ce problème, le module serializer nous permet de spécifier des groupes de propriétés d'objet à inclure dans la sérialisation. En spécifiant un groupe de propriétés, on peut éviter les boucles infinies en ne sérialisant que les propriétés pertinentes pour l'utilisation prévue. Ce qui nous permet d'éviter les références circulaires.

C. Interface d'une liste

Une fois l'affichage des listes fonctionnel, on a mis en place la fonctionnalité qui permet à l'utilisateur d'accéder à une liste en particulier et d'afficher certaines informations comme les 5 dernières opérations effectuées sur la liste et de qui elles proviennent.

```
/**
 * @Route("/api/liste/{id}", name="app_liste_id")
 */
public function liste(?Liste $liste): Response
{
    if (!$liste instanceof Liste)
    {
        return new JsonResponse( data: null, status: 400);
    }
    $data = $this->serializer->serialize($liste, format: 'json', ['groups' => 'liste']);

    return new JsonResponse($data, status: 200, [], json: 'json');
}

public function actionHistory(EntityManagerInterface $entityManager): Response
{
    $qb = $entityManager->createQueryBuilder('e');
    $qb->select( select: 'u.lastname as user_last_name', 'u.firstname as user_first_name', 'l.name as list_name', 'e.name as name', 'e.amount as amount')
    ->join( join: 'e.user', alias: 'u')
    ->join( join: 'e.liste', alias: 'l')
    ->setMaxResults( maxResults: 5);

    $result = $qb->getQuery()->getResult();

    return new JsonResponse($result, status: 200, [], json: 'json');
}
```

La réponse qui renverra la fonction est au format JSON et sera récupérée par le côté client sous forme de tableau d'objet de type Liste.

Du côté du front, on y affiche les dépenses cumulées de toutes les personnes affiliées à cette liste ainsi que tous les dépenses de l'utilisateur dans un même cadran. Le cadran du dessous lui permet d'afficher les 5 dernières opérations effectuées et par qui elles ont été faites. Enfin, en bas du deuxième cadran les deux boutons permettent d'ajouter ou d'enlever de l'argent.

Dépense total :	2983,19€
Mon coût total :	716,98€

Remboursement Payé par Kemil	176,98€
Remboursement Payé par Greg	716,26€
Péage 5 Payé par Julien	5,16€
Essence Payé par Tristan	76,48€
Péage 4 Payé par Julien	13,76€

+
AJOUTER UNE SOMME

-
ENLEVER UNE SOMME

D. Partage des listes

Maintenant que nous avons l'interface d'une liste, nous devons pouvoir la partager avec d'autres utilisateurs.

Pour pouvoir partager une liste, nous avons besoin de récupérer l'email de l'utilisateur avec qui la liste doit être partagée.

Il y a plusieurs vérifications qui vérifient qu'il n'y a pas de problème avec la liste (en cas de suppression par exemple) et l'autre servant à vérifier que l'utilisateur existe bien. Si les deux conditions sont vraies alors on va partager la liste et vérifier que l'utilisateur en question n'est pas déjà membre de celle-ci. Si tout est ok, la liste est partagée.

```
/**
 * @Route("/api/liste/{id}/share", methods={"POST"})
 */
public function shareList(?Liste $liste, Request $request, EntityManagerInterface $entityManager): JsonResponse
{
    $content = json_decode($request->getContent(), associative: true);
    $email = $content['username'];
    $user = $entityManager->getRepository(User::class)->findOneBy(['email' => $email]);

    if (!$liste) {
        return new JsonResponse(['message' => "La liste n'a pas été trouvé."], status: 404);
    }

    if (!$user) {
        return new JsonResponse(['message' => "Utilisateur inconnue."], status: 404);
    }

    $sharedList = $entityManager->getRepository(SharedList::class)->findOneBy(['liste' => $liste, 'user' => $user]);

    if ($sharedList) {
        return new JsonResponse(['message' => "La liste est déjà partagé avec cet utilisateur."], status: 400);
    }

    $sharedList = new SharedList();
    $sharedList->setListe($liste)
        ->setUser($user)
        ->setIsOwner(isOwner: false);

    $entityManager->persist($sharedList);
    $entityManager->flush();

    return new JsonResponse(['message' => "Liste partagé avec l'utilisateur."], status: 200);
}
```

D'un point de vue du front, le partage des listes fonctionnent de la même manière que leur création : un pop-up ou il suffit d'enregistrer l'adresse email de l'utilisateur voulu permet de lui partager la liste.

The screenshot displays the SmartInvest application interface. On the left, a list of expenses is shown with a total cost of 716,98€. The expenses include:

- Mon coût total : 716,98€
- Remboursement (Payé par Kemil) : 176,98€
- Remboursement (Payé par Greg) : 716,26€
- Péage 5 (Payé par Julien) : 5,16€
- Essence (Payé par Tristan) : 76,48€
- Péage 4 (Payé par Julien) : 13,76€

Below the list are buttons for '+ AJOUTER UNE SOMME' and '- ENLEVER UNE SOMME'. At the bottom left is a button '+ AJOUTER UNE LISTE'. On the right, a modal is open for sharing the list, titled 'Nom de l'utilisateur', with a text input field and two buttons: 'FERMER' and 'PARTAGER'.

E. Les opérations des listes

Venons maintenant à la dernière fonctionnalité mais aussi celle qui a été la plus complexe à mettre en place : les opérations des listes.

Au niveau du controller il n'y a rien de spécifique. Nous récupérons les informations du front-end, nous les transformons en valeurs pour une nouvelle opération, nous faisons appel à deux fonctions présentes dans notre entité liste, puis nous sauvegardons ces données.

```
/**
 * @Route("/api/liste/{id}/amount", name="app_amount")
 */
public function amount(EntityManagerInterface $entityManager, Request $request, Liste $liste): Response
{
    $content = json_decode($request->getContent(), associative: true);
    $expense = new Expense();
    $expense
        ->setAmount($content['amount'])
        ->setName($content['name'])
        ->setUser($this->getUser());

    $liste->addExpense($expense);
    $liste->updateTotalAmount();

    $entityManager->persist($liste);
    $entityManager->flush();

    return new JsonResponse(['message' => "Montant ajouté."], status: 200);
}
```

La petite complexité vient de nos deux fonctions de notre entité liste. J'ai tenté de mettre en place quelque chose mais ce n'était pas complet. Pour la première fonction, il me manquait la boucle foreach C'est mon collègue qui m'a indiqué cela, sinon le montant total est réinitialisé à chaque fois.

```
public function updateTotalAmount()
{
    $totalAmount = $this->getTotalAmount();

    foreach ($this->getExpenses() as $expense) {
        if ($expense->getAmount() > 0 ) {
            $totalAmount += $expense->getAmount();
        } else {
            $totalAmount -= $expense->getAmount();
        }
    }

    $this->setTotalAmount($totalAmount);
}
```

Pour la deuxième, j'avais bien mis en place mon tableau de dépense/ajout ainsi que la récupération de la liste comme juste ci-dessous.

```
public function addExpense(Expense $expense): self
{
    if (!$this->expenses->contains($expense)) {
        $this->expenses[] = $expense;
        $expense->setListe($this);
    }

    return $this;
}
```

Mais je n'avais pas défini ma variable comme étant une collection et je m'étais trompé dans le sens de ma liaison, donc forcément c'est en rencontrant les erreurs que j'ai pu corriger les différents problèmes.

```
/**
 * @var ArrayCollection|PersistentCollection $expenses
 * @Groups({"liste"})
 * @ORM\OneToMany(targetEntity=Expense::class, mappedBy="liste", orphanRemoval=true, cascade={"persist"})
 */
private $expenses;

public function __construct()
{
    $this->total_amount = 0; // Initialisez à zéro
    $this->expenses = new ArrayCollection();
}
```

```
/**
 * @return Collection<int, Expense>
 */
public function getExpenses(): Collection
{
    return $this->expenses;
}
```

3. Calcul du taux d'endettement

Revenus et charges pris en compte dans le calcul du taux d'endettement

Revenus mensuels

- ✓ le salaire net mensuel et les éventuelles primes régulières
- ✓ les rentes
- ✓ les éventuelles pensions perçues
- ✓ les aides sociales ou de l'Etat
- ✓ toute autre source de revenu régulier

Charges mensuelles

- ✓ le loyer ou la mensualité d'un éventuel prêt immobilier en cours
- ✓ le montant de la mensualité du futur crédit immo
- ✓ les éventuels autres crédits en cours (crédit conso par exemple)
- ✓ les charges incompressibles : électricité, gaz, internet, eau, abonnement transport, essence, assurances, etc.
- ✓ toutes les autres dépenses récurrentes : alimentation, pensions, etc.

CALCUL DU TAUX D'ENDETTEMENT

Revenus mensuelles
€

Charges mensuelles
€

Taux d'endettement

%

CALCULER

Nous avons développé un simulateur pour aider les utilisateurs à évaluer leur taux d'endettement. Nous avons précisé les différents types de revenus à inclure dans le revenu mensuel ainsi que les charges qui sont à prendre en compte, comme vous pouvez le voir sur la capture d'écran. Vous pouvez utiliser le simulateur en entrant votre revenu total dans le premier champ et vos charges mensuelles dans le second champ. Une fois que ces deux champs sont remplis, il suffit de cliquer sur le bouton « Calculer » pour obtenir le taux d'endettement. Cela donnera une estimation précise du niveau d'endettement actuel et aidera les utilisateurs à mieux planifier leurs dépenses futures.

4. Simulateur de crédit

Pourquoi réaliser une simulation de crédit ?

Soyez totalement autonome dans votre démarche de simulation de prêt. Pour cela, nous vous proposons un simulateur de crédit gratuit qui vous permet de faire une estimation de votre emprunt, hors assurance emprunteur. Crédit auto, crédit travaux, prêt personnel pour réaliser un projet, notre calculette vous aide à déterminer le montant de vos mensualités.

L'estimation de votre emprunt est essentielle pour vous permettre de trouver la formule la plus adaptée à vos projets et à votre budget. Réalisez une première simulation de crédit pour découvrir le montant de votre mensualité. Si celle-ci vous semble trop importante, vous pouvez allonger la durée du crédit. Dans le cas inverse, si votre capacité d'emprunt vous permet de rembourser une somme plus importante chaque mois, alors vous pouvez réduire la durée de votre crédit ou augmenter le montant de votre prêt.

Grâce à ce simulateur de crédit gratuit, outil simple d'utilisation, vous avez le pouvoir de bien emprunter pour concrétiser vos projets.

SIMULATEUR DE CRÉDITS

Capital emprunté

€

Période de remboursement

(en mois)

Taux d'intérêt

2,5 %

Mensualités

€

Intérêt

€

Cette page permet, comme son nom l'indique, de simuler un crédit. Il permet en indiquant le capital emprunté, la période de remboursement désiré et accessoirement le taux d'intérêt si ce dernier n'est pas à 2,5% d'avoir une estimation de l'intérêt qui devra être remboursé ainsi que les mensualités qui seront déversé sur la période donnée. Cette simulation n'est donnée qu'à titre indicatif et ne prend pas en compte les potentiels frais supplémentaire qui pourraient s'appliquer.

IV. Conclusion

Pendant développement du projet, comme vous avez pu le lire, on a rencontré un bon nombre de problématique qu'on a su surmonter tant bien que mal. Cela nous a permis d'acquérir de l'expérience, de gagner en connaissance et de développer nos compétences en matière de programmation web.

Bien que certaines fonctionnalités ne soient pas encore reliées au frontend, elles sont fonctionnelles et ont été testées avec Postman. Nous sommes convaincus que notre application répondra aux besoins de l'agence immobilière et de ses clients.

Il est important de noter que durant toute la réalisation du projet nous avons énormément communiquer sur les problèmes que chacun pouvait rencontrer. Cette communication nous a permis de tenir les délais fixés et de trouver des solutions rapidement. Chacun ayant une expérience et des compétences différentes, c'est un véritable partage de connaissances que nous avons effectué et c'est ce qui nous a permis de réaliser le projet tel qu'il est.

Bibliographie

<https://axios-http.com/fr/docs/intro>

<https://vuejs.org/>

<https://code.visualstudio.com/>

<https://www.jetbrains.com/fr-fr/phpstorm/>

<https://quasar.dev/>

<https://symfony.com/>

<https://www.typescriptlang.org/>

<https://developer.mozilla.org/fr/docs/Web/JavaScript>

<https://github.com/>

<https://filezilla-project.org/>

<https://www.postman.com/>

<https://www.php.net/manual/fr/index.php>

[Composer \(getcomposer.org\)](https://getcomposer.org)

[Penpot - Bring Design Freedom to your Product Team](#)